

Hibernate

While talking about the best Java web frameworks, Hibernate ORM can't be ignored. Hibernate is an essential object-relation mapping tool for Java programming language. It offers a mapping framework for a domain model (object-oriented) to one relational database. Hibernate can solve the object-relational impedance incongruity problems by substituting persistent and direct database with high-level object controlling functions. It is free software distributed under public 2.1 License of GNU Lesser General.

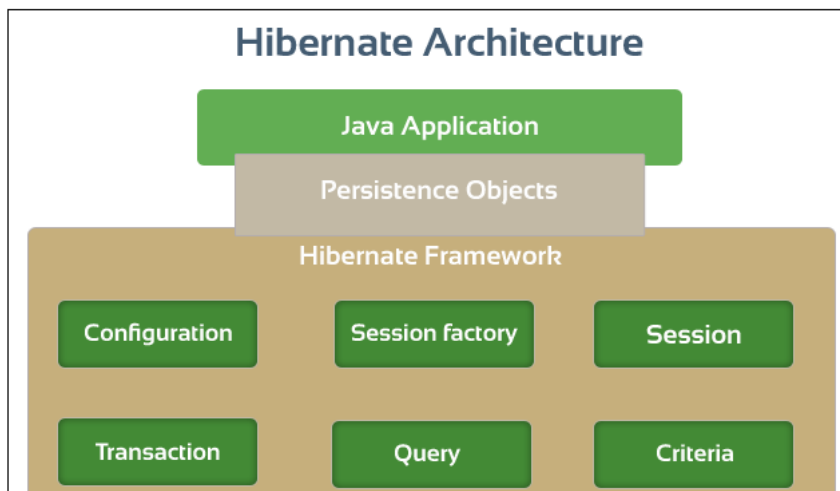
Pros:

- Hibernate enables you to communicate with any database by making tiny alternations in code
- MySQL, Db2 or Oracle, Hibernate is DB independent
- Caching instrument to bug catalog with same queries
- N+1 or Sluggish loading support
- Low risk of data loss and it requires less power

Cons:

- If power goes off, you can lose all your data
- Restarting can be extremely slow

Hibernate framework is an advanced ORM framework that lets you perform the database operation on Java objects. This framework carries the best features that help backend developers fine-tune data access layer.



Productivity-boosting features

- N+1 or Lazy loading support
- Built in Caching mechanism
- High-level object handling functions.
- It allows you to communicate with any database using tiny alterations.

Overview of Hibernate 5

Working with both Object-Oriented software and Relational Databases can be cumbersome and time consuming. Development costs are significantly higher due to a paradigm mismatch between how data is represented in objects versus relational databases. Hibernate is an Object/Relational Mapping solution for Java environments. The term Object/Relational Mapping refers to the technique of mapping data from an object model representation to a relational data model representation (and vice versa).

Hibernate not only takes care of the mapping from Java classes to database tables (and from Java data types to SQL data types), but also provides data query and retrieval facilities. It can significantly reduce development time otherwise spent with manual data handling in SQL and JDBC. Hibernates design goal is to relieve the developer from 95% of common data persistence-related programming tasks by eliminating the need for manual, hand-crafted data processing using SQL and JDBC. However, unlike many other persistence solutions, Hibernate does not hide the power of SQL from you and guarantees that your investment in relational technology and knowledge is as valid as always.

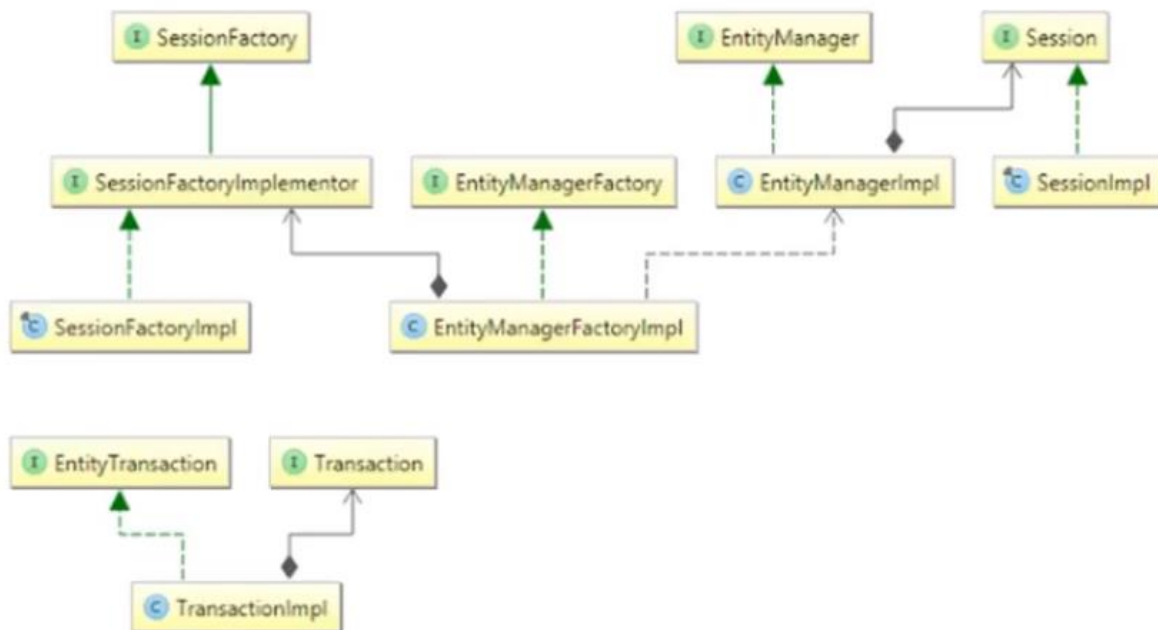
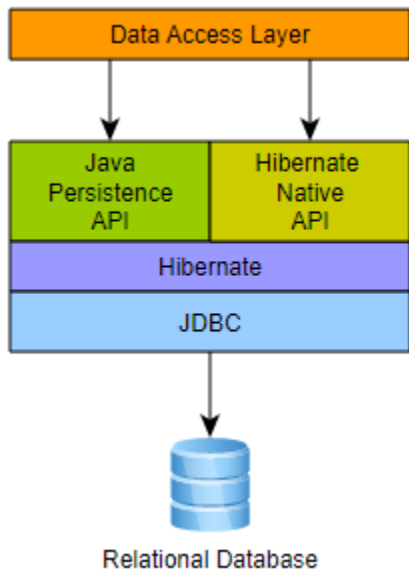
Hibernate May not be the best solution for data-centric applications that only use stored-procedures to implement the business logic in the database, it is most useful with object-oriented domain models and business logic in the Java-based middle-tier. However, Hibernate can certainly help you to remove or encapsulate vendor-specific SQL code and will help with the common task of result set translation from a tabular representation to a graph of objects.

System Requirements

Hibernate 5.2 and later versions require at least Java 1.8 and JDBC 4.2.

Architecture

Hibernate, as an ORM solution, effectively "sits between" the Java application data access layer and the Relational Database, as can be seen in the diagram above. The Java application makes use of the Hibernate APIs to load, store, query, etc its domain data.



As a JPA provider, Hibernate implements the Java Persistence API specifications and the association between JPA interfaces and Hibernate specific implementations can be visualized in the following diagram:

Hibernate Components

SessionFactory (org.hibernate.SessionFactory)

A thread-safe (and immutable) representation of the mapping of the application domain model to a database. Acts as a factory for **org.hibernate.Session** instances. The **EntityManagerFactory** is the JPA equivalent of a **SessionFactory** and basically those two converge into the same **SessionFactory** implementation.

A **SessionFactory** is very expensive to create, so, for any given database, the application should have only one associated **SessionFactory**. The **SessionFactory** maintains services that Hibernate uses across all Session(s) such as second level caches, connection pools, transaction system integrations, etc.

Session (org.hibernate.Session)

A single-threaded, short-lived object In JPA nomenclature, the Session is represented by an EntityManager. Behind the scenes, the Hibernate Session wraps a JDBC java.sql.Connection and acts as a factory for org.hibernate.Transaction instances. It maintains a generally "repeatable read" persistence context (first level cache) of the application domain model.

Transaction (org.hibernate.Transaction)

A single-threaded, short-lived object used by the application to demarcate individual physical transaction boundaries. EntityTransaction is the JPA equivalent and both act as an abstraction API to isolate the application from the underlying transaction system in use (JDBC or JTA).

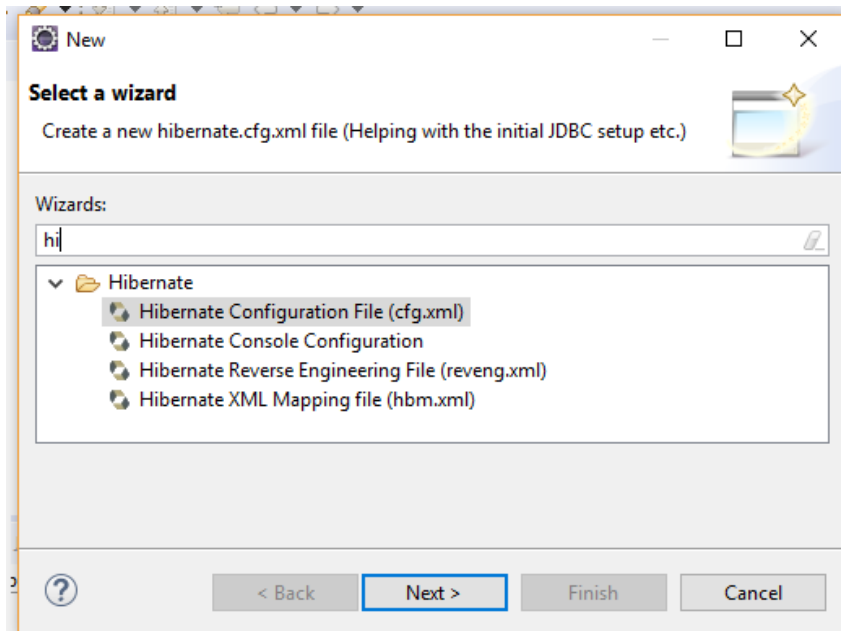
Installation JBoss Tools

Step 1: Goto Help on any version of eclipse and click Eclipse Marketplace and type **hibernate** to the search box

Step 2: Select JBoss Tols and install, it will ask you are you want to install plugin after some time than click on. Once you click ok, when installation complete it will prompt you to restart eclipse then click ok

Step 3: Once your eclipse restarts and plugin installed successfully you will see Hibernate as below from new project and other option and type **hibernate** as shown in the figure

Hibernate Annotations



Annotation	Package Detail/Import statement
<u>@Entity</u>	import javax.persistence.Entity;

<u>@Table</u>	import javax.persistence.Table;
<u>@Column</u>	import javax.persistence.Column;
<u>@Id</u>	import javax.persistence.Id;
<u>@GeneratedValue</u>	import javax.persistence.GeneratedValue;
<u>@OrderBy</u>	import javax.persistence.OrderBy;
<u>@Transient</u>	import javax.persistence.Transient;
<u>@Lob</u>	import javax.persistence.Lob;
<u>Hibernate Association Mapping Annotations</u>	
<u>@OneToOne</u>	import javax.persistence.OneToOne;
<u>@ManyToOne</u>	import javax.persistence.ManyToOne;
<u>@OneToMany</u>	import javax.persistence.OneToMany;
<u>@ManyToMany</u>	import javax.persistence.ManyToMany;
<u>@PrimaryKeyJoinColumn</u>	import javax.persistence.PrimaryKeyJoinColumn;
<u>@JoinColumn</u>	import javax.persistence.JoinColumn;
<u>@JoinTable</u>	import javax.persistence.JoinTable;
<u>@MapsId</u>	import javax.persistence.Id;

@Table

Specify the database table this Entity maps to using the name attribute of @Table annotation. In the example below, the data will be stored in 'company' table in the database.

```
@Entity
@Table(name = "company")
public class Company implements Serializable {
    ...
}
```

@Column

Specify the column mapping using @Column annotation to change the column name.

```
@Entity
@Table(name = "company")
public class Company implements Serializable {

    @Column(name = "name")
    private String name;

    ...
}
```

@Id

Annotate the id column using @Id for primary key field

```
@Entity
@Table(name = "company")
public class Company implements Serializable {

    @Id
    @Column(name = "id")
    private int id;

    ...
}
```

@GeneratedValue

Let database generate (auto-increment) the id column.

```
@Entity
@Table(name = "company")
public class Company implements Serializable {

    @Id
    @Column(name = "id")
```

```
@GeneratedValue(strategy=GenerationType.IDENTITY)
private int id;

...
}
```

@OrderBy

Sort your data using @OrderBy annotation. In example below, it will sort all contacts in a company by their firstname in ascending order.

```
@Entity
@Table(name = "company")
public class Company implements Serializable {

    @Id
    @Column(name = "id")
    @GeneratedValue
    private int id;

    @OrderBy("firstName asc")
    private Set contacts;

    .....
}
```

@Transient

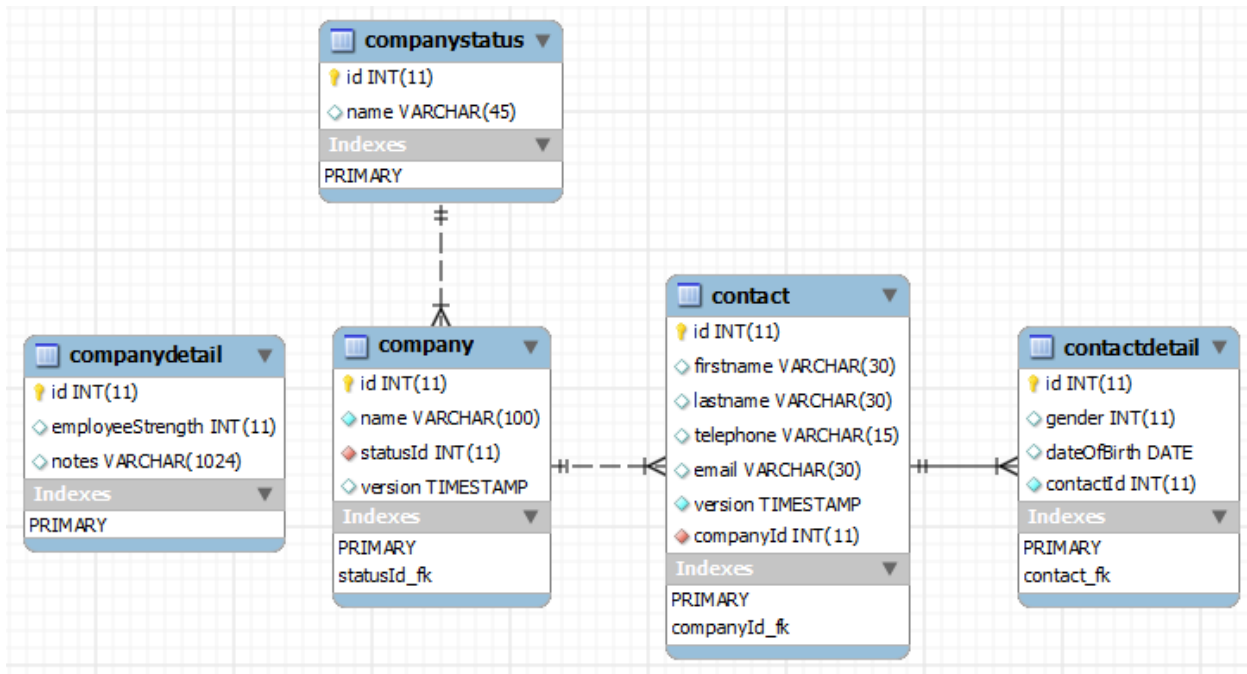
Annotate your transient properties with @Transient.

@Lob

Annotate large objects with @Lob.

Hibernate Association Mapping Annotations

Example App DB Schema



The database for this project is designed to illustrate various association mapping concepts. In RDBMS implementations, entities are joined using the following ways:

- Shared Primary Key
- Foreign Key
- Association Table

In our app,

- Tables company and companyDetail have shared values for primary key. It is a one-to-one association.
- Tables contact and contactDetail are linked through a foreign key. It is also a one to one association.
- Tables contact and company are linked through a foreign key in many-to-one association with contact being the owner.
- Tables company and companyStatus are linked through a foreign key in many-to-one association with company being the owner.

@OneToOne

- Use @PrimaryKeyJoinColumn for associated entities sharing the same primary key.
- Use @JoinColumn & @OneToOne mappedBy attribute when foreign key is held by one of the entities.

- Use @JoinTable and mappedBy entities linked through an association table.
- Persist two entities with shared key using @MapsId
- For entities Company and CompanyDetail sharing the same primary key, we can associate them using @OneToOne and @PrimaryKeyJoinColumn as shown in the example below.

Notice that the id property of CompanyDetail is NOT annotated with @GeneratedValue. It will be populated by id value of Company.

```
@Entity
@Table(name = "company")
public class Company implements Serializable {

    @Id
    @Column(name = "id")
    @GeneratedValue
    private int id;

    @OneToOne(cascade = CascadeType.MERGE)
    @PrimaryKeyJoinColumn
    private CompanyDetail companyDetail;

    ...
}

@Entity
@Table(name = "companyDetail")
public class CompanyDetail implements Serializable {

    @Id
    @Column(name = "id")
    private int id;

    ...
}
```

For entities Contact and ContactDetail linked through a foreign key, we can use @OneToOne and @JoinColumn annotations. In example below, the id generated for Contact will be mapped to 'contact_id' column of ContactDetail table. Please note the usage of @MapsId for the same.

```
@Entity
@Table(name = "contactDetail")
public class ContactDetail implements Serializable {

    @Id
    @Column(name = "id")
    @GeneratedValue
    private int id;

    @OneToOne
    @MapsId
    @JoinColumn(name = "contactId")
    private Contact contact;

    ...
}

@Entity
@Table(name = "contact")
public class Contact implements Serializable {

    @Id
    @Column(name = "ID")
    @GeneratedValue
    private Integer id;

    @OneToOne(mappedBy = "contact", cascade = CascadeType.ALL)
    private ContactDetail contactDetail;

    ....
}
```

Also note that the relationship between Company and CompanyDetail is uni-directional. On the other hand, the relationship between Contact and Contact Detail is bi-directional and that can be achieved using 'mappedBy' attribute.

The rationale to have one relationship as uni-directional and other as bi-directional in this tutorial is to illustrate both concepts and their usage. You can opt for uni-directional or bi-directional relationships to suit your needs.

@ManyToOne

Use @JoinColumn when foreign key is held by one of the entities.

Use @JoinTable for entities linked through an association table.

The two examples below illustrate many-to-one relationships. Contact to Company and Company to CompanyStatus. Many contacts can belong to a company. Similarly many companies can share the same status (Lead, Prospect, Customer) - there will be many companies that are currently leads.

```
@Entity
@Table(name = "contact")
public class Contact implements Serializable {

    @ManyToOne
    @JoinColumn(name = "companyId")
    private Company company;

    ...

}
```

```
@Entity
@Table(name = "company")
public class Company implements Serializable {

    @ManyToOne
    @JoinColumn(name = "statusId")
    private CompanyStatus status;

    ...

}
```

@OneToMany

Use mappedBy attribute for bi-directional associations withManyToOne being the owner.

OneToMany being the owner or unidirectional with foreign key - try to avoid such associations but can be achieved with @JoinColumn

@JoinTable for Unidirectional with association table

Please see the many-to-one relationship between Contact and Company above. Company to Contact will be a one-to-many relationship. The owner of this relationship is Contact and hence we will use 'mappedBy' attribute in Company to make it bi-directional relationship.

@Entity

@Table(name = "company")

public class Company implements Serializable {

 @OneToMany(mappedBy = "company", fetch = FetchType.EAGER)

 @OrderBy("firstName asc")

 private Set contacts;

 ...

}

Again, for this tutorial, we have kept Company to CompanyStatus relationship as uni-directional.

@ManyToMany

Use @JoinTable for entities linked through an association table.

Use mappedBy attribute for bi-directional association.

@PrimaryKeyJoinColumn

@PrimaryKeyJoinColumn annotation is used for associated entities sharing the same primary key.

See [OneToOne](#) section for details.

@Entity

@Table(name = "company")

public class Company implements Serializable {

```

@Id
@Column(name = "id")
@GeneratedValue
private int id;

@OneToOne(cascade = CascadeType.MERGE)
@PrimaryKeyJoinColumn
private CompanyDetail companyDetail;

...
}

```

@JoinColumn

Use @JoinColumn annotation for one-to-one or many-to-one associations when foreign key is held by one of the entities. We can use @OneToOne or @ManyToOne mappedBy attribute for bi-directional relations. Also see [OneToOne](#) and [ManyToOne](#) sections for more details.

```

@ManyToOne
@JoinColumn(name = "statusId")
private CompanyStatus status;

```

@JoinTable

Use @JoinTable and mappedBy for entities linked through an association table.

@MapsId

Persist two entities with shared key (when one entity holds a foreign key to the other) using @MapsId annotation. See [OneToOne](#) section for details.

```

@OneToOne
@MapsId
@JoinColumn(name = "contactId")
private Contact contact;

```

Java Based Session Factory without hibernate.cfg.xml file

```

package com.utils;
import java.util.HashMap;
import java.util.Map;

```

```
import org.hibernate.SessionFactory;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistry;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;
import org.hibernate.cfg.Environment;

public class HibernateUtilsJava {
    private static StandardServiceRegistry standardServiceRegistry;
    private static SessionFactory sessionFactory;
    static{

        StandardServiceRegistryBuilder standardServiceRegistryBuilder = new
StandardServiceRegistryBuilder();
        Map<String, String> settings = new HashMap<>();
        settings.put(Environment.URL,
"jdbc:mysql://localhost:3306/hibernatedemo");
        settings.put(Environment.USER, "root");
        settings.put(Environment.PASS, "");
        settings.put(Environment.DRIVER, "com.mysql.jdbc.Driver");
        settings.put(Environment.DIALECT,
"org.hibernate.dialect.MySQL5Dialect");
        settings.put(Environment.HBM2DDL_AUTO, "create");
        settings.put(Environment.SHOW_SQL, "true");
        settings.put(Environment.FORMAT_SQL, "true");
        standardServiceRegistryBuilder.applySettings(settings);

        standardServiceRegistry = standardServiceRegistryBuilder.build();
        MetadataSources metadataSource = new
MetadataSources(standardServiceRegistry);
        metadataSource.addAnnotatedClass(Student.class);
        Metadata metadata = metadataSource.getMetadataBuilder().build();

        sessionFactory = metadata.getSessionFactoryBuilder().build();
    }

    public static SessionFactory getSessionFactory(){
        return sessionFactory;
    }
}
```

Writing Hibernate Testing Class

```
import org.hibernate.Session;

import com.utils.HibernateUtils;

public class Demo {
    public static void main(String[] args) {

        try(Session session = HibernateUtils.getSessionFactory().openSession()){
            String sql="select version();";
            String result = (String)
session.createNativeQuery(sql).getSingleResult();
            System.out.println(result);
        }
    }
}
```

```

    }catch(Exception e){
        System.out.println("Error: "+e.getMessage());
    }
}

```

Writing configuration file

hibernate.cfg.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property
name="hibernate.connection.driver_class">com.mysql.jdbc.Driver</property>
        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/hibernatedemo</property>
        <property name="hibernate.connection.username">root</property>
        <property
name="hibernate.dialect">org.hibernate.dialect.MySQL5Dialect</property>

        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>
        <property name="hibernate.connection.pool_size">2</property>

        <mapping class="com.model.DemoEntity" />
    </session-factory>
</hibernate-configuration>

```

Writing Entity Model Class

```

package com.model;

import java.io.Serializable;

import javax.persistence.Column;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

import org.hibernate.annotations.DynamicUpdate;
@Entity
@Table(name="tbl_profiles")
@DynamicUpdate
public class Profile implements Serializable {

    private static final long serialVersionUID = 1L;
    @Id
    @GeneratedValue(strategy= GenerationType.AUTO)
    @Column(name="ORG_ID")
    private int id;

```



```

        @Column(name="ORG_NAME",    length=50,    nullable    =    false,    unique=true,
updatable=false)
        private String organizationName;
        @Column(name="ADDRESS")
        private String address;
        @Column(name="EMAIL_ADDRESS")
        private String emailAddress;
        @Column(name="CONTACT_TELEPHONE")
        private String contactTelephone;
        @Column(name="CONTACT_PERSON")
        private String contactPerson;
        @Column(name="CONTACT_PERSON_MOBILE")
        private String contactPersonMobile;
        @Column(name="DESCRIPTION")
        private String description;
        @Column(name="STATUS")
        private int status;

        public Profile(){

        }

        public Profile(int id, String organizationName, String address, String
emailAddress, String contactTelephone,
                        String contactPerson, String contactPersonMobile, String
description, int status) {
            this.id = id;
            this.organizationName = organizationName;
            this.address = address;
            this.emailAddress = emailAddress;
            this.contactTelephone = contactTelephone;
            this.contactPerson = contactPerson;
            this.contactPersonMobile = contactPersonMobile;
            this.description = description;
            this.status = status;
        }
        public Profile(String organizationName, String address, String emailAddress,
String contactTelephone,
                        String contactPerson, String contactPersonMobile,String
description, int status) {
            this.organizationName = organizationName;
            this.address = address;
            this.emailAddress = emailAddress;
            this.contactTelephone = contactTelephone;
            this.contactPerson = contactPerson;
            this.contactPersonMobile = contactPersonMobile;
            this.description = description;
            this.status = status;
        }

        public int getId() {
            return id;
        }

        public void setId(int id) {

```

```
        this.id = id;
    }

    public String getOrganizationName() {
        return organizationName;
    }

    public void setOrganizationName(String organizationName) {
        this.organizationName = organizationName;
    }

    public String getAddress() {
        return address;
    }

    public void setAddress(String address) {
        this.address = address;
    }

    public String getEmailAddress() {
        return emailAddress;
    }

    public void setEmailAddress(String emailAddress) {
        this.emailAddress = emailAddress;
    }

    public String getContactTelephone() {
        return contactTelephone;
    }

    public void setContactTelephone(String contactTelephone) {
        this.contactTelephone = contactTelephone;
    }

    public String getContactPerson() {
        return contactPerson;
    }

    public void setContactPerson(String contactPerson) {
        this.contactPerson = contactPerson;
    }

    public String getContactPersonMobile() {
        return contactPersonMobile;
    }

    public void setContactPersonMobile(String contactPersonMobile) {
        this.contactPersonMobile = contactPersonMobile;
    }

    public int getStatus() {
        return status;
    }
}
```

```

public void setStatus(int status) {
    this.status = status;
}

public void setDescription(String description){
    this.description = description;
}

public String getDescription(){
    return description;
}

@Override
public String toString() {
    return "Profile [id=" + id + ", organizationName=" + organizationName +
    ", address=" + address
    + ", emailAddress=" + emailAddress + ", contactTelephone="
+ contactTelephone + ", contactPerson="
    + contactPerson + ", contactPersonMobile="
+ contactPersonMobile + ", status=" + status + "]\n";
}
}

```

Writing Hibernate Connection and Factory class

```

package com.utils;

import org.hibernate.SessionFactory;
import org.hibernate.boot.Metadata;
import org.hibernate.boot.MetadataSources;
import org.hibernate.boot.registry.StandardServiceRegistry;
import org.hibernate.boot.registry.StandardServiceRegistryBuilder;

public class HibernateUtils {
    private static StandardServiceRegistry standardServiceRegistry;
    private static SessionFactory sessionFactory;

    static{
        try {
            if(sessionFactory==null){
                standardServiceRegistry = new
                StandardServiceRegistryBuilder().configure().build();
                MetadataSources metadataSource = new
                MetadataSources(standardServiceRegistry);
                Metadata metadata = metadataSource
                metadataSource.getMetadataBuilder().build();
                sessionFactory = metadata.getSessionFactoryBuilder().build();
            }
        } catch (Exception e) {
            if(standardServiceRegistry!=null){
                StandardServiceRegistryBuilder.destroy(standardServiceRegistry);
            }
        }
    }
}

```

```

    }

    public static SessionFactory getSessionFactory(){
        return sessionFactory;
    }
}

```

Writing CRUD functionality interface class

```

package com.dao;

import com.model.Profile;

public interface ProfileDAO {

    int create(Profile prfile);

    Profile read(Integer id);

    void update(Profile profile);

    void delete(Profile profile);
}

```

Writing CRUD implementation class

```

package com.dao.impl;

import org.hibernate.HibernateException;
import org.hibernate.Session;

import com.dao.ProfileDAO;
import com.model.Profile;
import com.utils.HibernateUtils;

public class ProfileDAOImpl implements ProfileDAO {

    @Override
    public int create(Profile profile) {
        try(Session session = HibernateUtils.getSessionFactory().openSession()){
            session.beginTransaction();
            Integer id = (Integer) session.save(profile);
            session.getTransaction().commit();
            return id;
        }catch(HibernateException e){
            System.out.println("Error: "+e.getMessage());
        }
        return 0;
    }

    @Override
    public Profile read(Integer id) {
        try(Session session = HibernateUtils.getSessionFactory().openSession() ){
            //hibernate reading method
            return session.get(Profile.class, id);
        }
    }
}

```

```

    }catch(Exception e){
        System.out.println("Reading Error: "+e.getMessage());
    }
    return null;
}

@Override
public void update(Profile profile) {
    try(Session session = HibernateUtils.getSessionFactory().openSession()){
        session.beginTransaction();
        //hibernate update
        session.update(profile);
        session.getTransaction().commit();
        System.out.println("Record Update Successfully");
    }catch(HibernateException e){
        System.out.println("Error: "+e.getMessage());
    }
}

@Override
public void delete(Profile profile) {
    try(Session session = HibernateUtils.getSessionFactory().openSession()){
        //reading profile
        session.beginTransaction();
        //hibernate delete method
        session.delete(profile);
        session.getTransaction().commit();
        System.out.println("Record Deleted Successfully");

    }catch(Exception e){
        System.out.println("Delete Error: "+e.getMessage());
    }
}
}
}

```

Writing client program for CRUD functionality

```

package com.client;

import com.dao.ProfileDAO;
import com.dao.impl.ProfileDAOImpl;
import com.model.Profile;

public class Demo {
    public static void main(String[] args) {
        ProfileDAO profileDAO = new ProfileDAOImpl();

        Profile profile = new Profile("NAST COLLETE", "DHANGADHI",
"info@nast.edu.np", "091-523320", "Bhatta", "76543", "543", 1);
        if(profileDAO.create(profile)>0){
            System.out.println("Saved Successfully");
        }else{
            System.out.println("Unable to Save!");
        }
    }
}

```

```

    }

    //Profile profile = profileDAO.read(1);
    //System.out.println(profile.getOrganizationName()+ " AND
"+profile.getContactPerson());

    //update
    //      int id =1;
    //      Profile oldProfile = profileDAO.read(id);
    //      if(oldProfile!=null){
    //          oldProfile.setContactPerson("Mahesh Bhatta");
    //          oldProfile.setEmailAddress("info@mb.com.np");
    //          profileDAO.update(oldProfile);
    //      }else{
    //          System.out.println("Record of ID: "+id+" is not
found");
    //      }

//      int id =1;
//      Profile oldProfile = profileDAO.read(id);
//      if(oldProfile!=null){
//          profileDAO.delete(oldProfile);
//      }else{
//          System.out.println("Record of ID: "+id+" is not found");
//      }
}
}

```

pom.xml file for desktop app

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>np.com.coretechnology</groupId>
    <artifactId>desktop</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <packaging>jar</packaging>

    <name>desktop</name>
    <url>http://maven.apache.org</url>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>

    <dependencies>

        <dependency>
            <groupId>org.hibernate</groupId>

```

```

        <artifactId>hibernate-core</artifactId>
        <version>5.4.0.Final</version>
    </dependency>

    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>5.1.25</version>
    </dependency>

    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>3.8.1</version>
        <scope>test</scope>
    </dependency>

</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.6.1</version>
            <configuration>
                <source>1.8</source>
                <target>1.8</target>
            </configuration>
        </plugin>
    </plugins>
</build>
</project>

```

pom.xml file for web application

```

<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>np.com.coretechnology</groupId>
    <artifactId>web</artifactId>
    <packaging>war</packaging>
    <version>0.0.1-SNAPSHOT</version>
    <name>web Maven Webapp</name>
    <url>http://maven.apache.org</url>
    <dependencies>

        <dependency>
            <groupId>javax.servlet</groupId>
            <artifactId>javax.servlet-api</artifactId>
            <version>3.1.0</version>
            <scope>provided</scope>
        </dependency>
    </dependencies>

```

```
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.4.0.Final</version>
</dependency>

<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>5.1.25</version>
</dependency>

<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>3.8.1</version>
  <scope>test</scope>
</dependency>
</dependencies>
<build>
  <finalName>web</finalName>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.6.1</version>
      <configuration>
        <source>1.8</source>
        <target>1.8</target>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>
```

Questions

Define Hibernate. Explain with Architecture Diagram

Define Annotations. Explain Different types Annotations with examples

Define Maven and Gradle. Explain the working of Maven and Gradle and also its advantages

Define pom.xml file. Illustrate with example for desktop and web application